management problem in and of itself. How can you have multiple programs on one machine that might use different versions of a gem? Or, because gems change all the time, how can you guarantee that everybody working on the application will be using the same set of gems.

The answer to both of these questions and many more is Bundler.[1] Bundler is itself a Ruby gem, and it manages a manifest file of the gems and versions in use on a project. Bundler allows you to specify the versions in use, and limit your Ruby programs to only find the specific version of a gem endorsed by Bundler.

In order to use Bundler, you first must install it. It's a Ruby gem, so `gem install bundler` will do it for you. The installation gives you the `bundler` command-line program, which you then use to manage your gems.

Building a Gemfile

To use Bundler effectively, you create a listing of all the gems in your application in a file called `Gemfile`. The `Gemfile` should be in the root directory of your application.

Here's a small sample, which happens to be the Gemfile in use for the code examples and code management for this very book, at least, so far:

```
source "https://rubygems.org"

ruby "3.2.0"

gem "debug"
gem "i18n"
gem "nokogiri"
gem "rspec"
gem "solargraph"
gem "standardrb"
```

The Gemfile is made up of a series of different statements that describe the set of all the gems being used by the project. The Gemfile is actually Ruby code, which means that each line is actually a Ruby method call (without parentheses), and also means that Ruby syntax will be used as the Gemfile gets more complicated.

The first line declares a `source`, we're using `"https://rubygems.org"`, which is the central repository for RubyGems, but you could also create your own server and use that as a source. Many companies use internal gem servers to distribute common internal code that is not public. You can declare multiple sources, but the recommendation is to have one top-level source and limit the scope of secondary sources. (You'll see how to do that in a moment.)

The next line specifies the Ruby version that this application expects. As we saw in the previous section, Ruby Gems are matched to specific versions of Ruby and stored separately for different versions. This line of code specifies an exact version of Ruby—unlike gem declarations the Ruby version must be exact.

After that, we declare gems. Each gem declaration is the word `gem` followed by the official name of the gem. The order of the gem listings doesn't matter, though it's usually considered polite to keep them alphabetical.

That's a pretty basic Gemfile, as you'll see shortly, they can get more complicated.

1. http://bundler.io